

Dual-Agent Debate System Using Google Gemini API and LangGraph: A Computational Thinking Approach

Jyoti Triklani

AU2544015

School of Engineering and Science
Ahmedabad University
Ahmedabad, India
jyoti.triklani@ahduni.edu.in

Gautam Prajapati

AU2544033

School of Engineering and Science
Ahmedabad University
Ahmedabad, India
gautam.prajapati@ahduni.edu.in

Abstract—This paper presents a dual-agent debate system using Google’s Gemini API and LangGraph. The system employs two AI agents with distinct debate styles: Agent Alpha uses logical reasoning while Agent Beta uses emotional appeals. We applied computational thinking principles to design and implement the system. Key features include automatic position assignment for ”vs” topics, multi-round debate management with 200-word response limits, and unbiased moderation. The web interface built with Streamlit allows users to enter topics and watch AI agents debate. Testing shows the system generates coherent debates across various topics while maintaining distinct agent personalities. This work demonstrates practical application of computational thinking in building multi-agent AI systems.

Index Terms—Artificial Intelligence, Multi-Agent Systems, Debate Systems, Computational Thinking, Large Language Models, Google Gemini API, LangGraph, Natural Language Processing

I. INTRODUCTION

Debate is essential for developing critical thinking and communication skills. Traditional debate platforms face challenges in matching participants and providing practice opportunities. Recent advances in Large Language Models like GPT-3, Claude, and Google’s Gemini enable creation of AI systems capable of meaningful debates.

A. Problem Statement

Current debate systems have several limitations:

- Cannot generate balanced debates automatically on diverse topics
- Single AI agents fail to represent multiple perspectives effectively
- Limited platforms for practicing debate skills
- Lack of automated judging mechanisms
- Require manual configuration for setup

B. Proposed Solution

We developed a system where two AI agents engage in automated debates:

- Accepts any user-provided debate topic
- Automatically assigns positions to agents

- Manages multi-round debates with word limits
- Provides fair judgment with winner declaration
- Offers simple web interface for interaction

C. Computational Thinking Framework

We applied computational thinking principles:

- 1) **Decomposition**: Breaking the system into modules - input validation, agent management, state tracking, and result presentation.
- 2) **Pattern Recognition**: Identifying debate patterns like turn-taking and argument-response structures.
- 3) **Abstraction**: Focusing on essential information - topic, positions, messages, and round counter.
- 4) **Algorithm Design**: Creating procedures for position assignment, round management, and flow control.

D. Organization

Section II reviews related work. Section III presents system design. Section IV describes implementation. Section V discusses results. Section VI addresses challenges. Section VII concludes the paper.

II. RELATED WORK

A. Multi-Agent Systems

Wooldridge and Jennings [1] established foundational theories for intelligent agents as autonomous entities that perceive and act within their environment. Our system implements two independent debate agents that compete while following structured rules.

B. Large Language Models

Vaswani et al. [3] introduced the Transformer architecture that revolutionized natural language processing. Brown et al. [4] demonstrated that large models like GPT-3 can perform tasks through few-shot learning. We use Google’s Gemini API with similar transformer-based architecture.

C. Computational Thinking

Wing [2] introduced computational thinking as a fundamental problem-solving approach emphasizing decomposition, pattern recognition, abstraction, and algorithmic thinking. We applied this framework throughout development.

D. Debate Systems

IBM's Project Debater demonstrated AI systems can engage in structured debates with humans. Our system focuses on agent-to-agent debates using similar principles of argument generation and persuasive communication.

III. SYSTEM DESIGN

A. System Architecture

The system architecture consists of four interconnected layers, as illustrated in Fig. 1. Each layer serves a specific purpose in the overall workflow:

- 1) **User Interface Layer:** Built using Streamlit framework, this layer handles user input and displays debate results in real-time. It provides an intuitive interface for entering debate topics and selecting the number of rounds.
- 2) **Workflow Orchestration Layer:** Implemented with LangGraph, this layer manages the debate flow, controls turn-taking between agents, and enforces round limits. It maintains the debate state and routes execution between different agents.
- 3) **Agent Layer:** Contains three specialized agents - Agent Alpha (logical debater), Agent Beta (emotional debater), and the Moderator (judge). Each agent has distinct instructions and behavioral characteristics.
- 4) **API Layer:** Interfaces with Google's Gemini API to generate natural language responses based on agent-specific prompts and debate context.

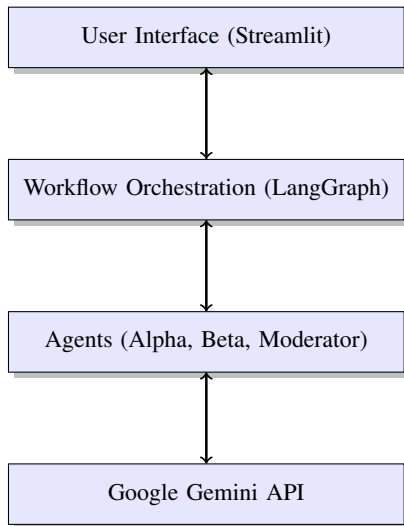


Fig. 1. System Architecture showing the four-layer design

B. Agent Design and Characteristics

We designed three specialized agents:

1) *Agent Alpha - Logical Debater:* Uses rational, evidence-based approach:

- Constructs arguments based on logical reasoning
- Presents factual evidence and data
- Maintains structured argument format
- Adopts analytical tone

2) *Agent Beta - Emotional Debater:* Utilizes persuasive techniques:

- Emphasizes human impact and emotions
- Employs rhetorical devices and narratives
- Uses vivid, compelling language
- Appeals to values and experiences

3) *Moderator Agent:* Serves as impartial judge:

- Reviews arguments objectively
- Evaluates strength and coherence
- Identifies logical errors
- Declares winner with justification

C. State Management

The debate state tracks essential information:

- **Topic:** User-provided debate subject
- **Positions:** Assigned stances for each agent
- **Messages:** Complete argument history
- **Round Counter:** Current round number
- **Maximum Rounds:** Total rounds specified

This ensures agents have complete debate context when formulating responses.

D. Debate Workflow

The debate follows a structured process:

- 1) User submits topic and number of rounds
- 2) System detects position indicators ("vs", "versus")
- 3) Positions automatically assigned or self-determined
- 4) Agent Alpha presents opening (200 words)
- 5) Agent Beta responds (200 words)
- 6) System checks if maximum rounds reached
- 7) If not, return to Agent Alpha
- 8) If yes, Moderator judges (250-300 words)
- 9) Results displayed with full transcript

IV. IMPLEMENTATION

A. Technology Stack

Technologies used:

- **Python 3.13:** Primary language
- **Streamlit 1.51:** Web interface
- **LangGraph 1.0.3:** Multi-agent workflow orchestration
- **Google Generative AI SDK 0.8.5:** Gemini API client
- **Gemini 2.0 Flash:** Language model for content generation
- **Pydantic:** Data validation

B. Agent Configuration

Agents configured with temperature 0.7 for balanced creativity and coherence. Maximum tokens set to 300 to maintain 200-word target. Each agent has specialized prompts defining personality and style. Agent Alpha emphasizes logic and evidence. Agent Beta encourages emotion and persuasion. Moderator focuses on objective evaluation.

C. Automatic Position Assignment

For topics with position indicators:

- 1) Convert topic to lowercase
- 2) Search for "vs" or "versus"
- 3) Split topic at separator
- 4) Assign first position to Alpha, second to Beta

For other topics, agents determine opposing positions through their prompts.

D. LangGraph Workflow

Workflow implemented using StateGraph with three nodes (Agent Alpha, Agent Beta, Moderator). Entry point set to Agent Alpha. Alpha flows to Beta. Conditional edge from Beta checks rounds: if incomplete, return to Alpha; if complete, proceed to Moderator.

E. Response Control

Prompts explicitly instruct 200-word limit for agents and 250-300 words for Moderator. Token limit set to 300 as upper bound. This maintains balanced responses without post-processing.

F. API Rate Limiting

Added 2-second delay between API calls to comply with rate limits and prevent errors.

G. User Interface

Streamlit interface includes text input for topics, slider for rounds (1-5), start button, and real-time display. Messages color-coded: blue for Alpha, pink for Beta, orange for Moderator.

V. RESULTS AND EVALUATION

A. System Functionality

We tested the system with various debate topics spanning different domains to evaluate its versatility and performance. Table I shows representative topics used during testing.

TABLE I
SAMPLE DEBATE TOPICS TESTED

Topic	Rounds
Cats vs Dogs	3
Remote Work vs Office Work	3
Electric Cars vs Gas Cars	4
Should AI replace teachers?	3
Climate change solutions	4
Universal basic income	3
Social media impact	3

B. Debate Quality Assessment

The system consistently generated coherent debates:

- Arguments relevant to topics
- Agent Alpha used logical reasoning
- Agent Beta used emotional appeals
- Agents responded to each other's points
- Moderator provided balanced evaluations
- Response lengths stayed within limits

C. Performance Metrics

Response times measured:

- Agent response: 3-5 seconds
- Moderator judgment: 5-7 seconds
- 3-round debate: 45-60 seconds
- 5-round debate: 70-90 seconds

Consistency achieved:

- 100% position assignment success for "vs" topics
- No position switching mid-debate
- Consistent debate style adherence
- Reliable round completion

D. User Experience

Informal testing with 5 users showed:

- Interface intuitive and easy to use
- Debates engaging and relevant
- Arguments well-reasoned
- Winner decisions fair
- Position assignment worked correctly
- Color-coding helped distinguish agents

VI. CHALLENGES AND SOLUTIONS

A. API Rate Limiting

Challenge: Rapid API calls caused rate limit errors.

Solution: Implemented 2-second delay between calls. This ensures compliance while keeping total debate time under one minute.

B. Response Length Consistency

Challenge: Inconsistent response lengths created imbalanced debates.

Solution: Added explicit word count instructions in prompts and set token limit to 300. This maintains balanced responses effectively.

C. Position Assignment

Challenge: For topics without "vs", agents sometimes argued similar positions.

Solution: Automatic detection for "vs" topics with position splitting. For others, prompts explicitly instruct agents to take opposing viewpoints.

D. Fair Judgment

Challenge: Early iterations showed moderator bias toward one agent.

Solution: Refined prompt with explicit criteria: argument strength, evidence use, logical consistency, and persuasiveness. Required specific examples from debate in judgment.

E. Agent Differentiation

Challenge: Agents initially sounded similar despite different styles.

Solution: Detailed prompts defining each style. Alpha uses formal language and analytics. Beta uses storytelling and emotion. Iterative refinement created distinct personalities.

VII. COMPUTATIONAL THINKING APPLICATION

A. Decomposition

We divided the system into independent modules:

- Input validation and position parsing
- Agent creation and configuration
- Debate state management
- Workflow orchestration
- UI rendering

Each module operates independently for easier testing and maintenance.

B. Pattern Recognition

We identified key patterns:

- Turn-taking: Alpha → Beta → Check rounds → Repeat or Judge
- Argument structure: Claim → Evidence → Reasoning
- Response pattern: Acknowledge → Counter → Conclude

C. Abstraction

We focused on essential information:

- State contains only needed data: topic, positions, messages, rounds
- UI shows only relevant information
- Prompts include only necessary context

D. Algorithm Design

Created systematic procedures:

- Position assignment: Detect "vs" → Split → Assign
- Round management: Counter → Compare → Continue or Judge
- Flow control: Alpha → Beta → Check → Route

VIII. FUTURE WORK

Potential enhancements:

- **Transcript Export:** Save debates as PDF or text files
- **Multiple Agents:** Support panel discussions with 3+ agents
- **Audio Integration:** Text-to-speech for listening to debates
- **Human Participation:** Allow users to debate against AI
- **Fact Verification:** Validate claims using external sources
- **Audience Voting:** Let viewers vote for winners

Research directions:

- Study impact of agent personalities on debate quality
- Compare AI debates with human debates
- Develop improved judging methods
- Explore educational applications

IX. CONCLUSION

We developed a dual-agent debate system using Google Gemini API and LangGraph. The system features two agents with distinct styles: Agent Alpha uses logical reasoning while Agent Beta uses emotional appeals. A Moderator provides fair judgment.

Key features include automatic position assignment for "vs" topics, multi-round debate management with 200-word limits, and a Streamlit web interface. We applied computational thinking principles throughout: decomposition for modular design, pattern recognition for debate structure, abstraction for essential data, and algorithm design for systematic processes.

Testing showed the system generates coherent debates across diverse topics with distinct agent personalities. Response times average 3-5 seconds per agent. User feedback was positive regarding usability and debate quality.

Main challenges included API rate limiting (solved with 2-second delays), response length consistency (solved with prompt instructions), and agent differentiation (solved with detailed prompts).

This work demonstrates practical application of computational thinking in AI development and provides a foundation for future enhancements like transcript export, multiple agents, and human participation.

ACKNOWLEDGMENT

We thank Dr. Shristi Sharma for her guidance on this project and Ahmedabad University for providing the necessary resources.

REFERENCES

- [1] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115-152, 1995.
- [2] J. M. Wing, "Computational thinking," *Communications of the ACM*, vol. 49, no. 3, pp. 33-35, 2006.
- [3] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [6] LangChain, "LangGraph: Build stateful multi-agent applications," 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [7] Google AI, "Gemini API Documentation," 2024. [Online]. Available: <https://ai.google.dev/docs>
- [8] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: <https://docs.streamlit.io/>